



4

10

1500 sp

عملي

كلية الهندسة المعلوماتية

السنة الأولى

تتمة الحلقات التكرارية + المصفوفات

د. ماهر صارم



محتوى مجاني غير مخصص للبيع التجاري

RB Informatcs; 17/12/2025

برمجة ا

في المحاضرة السابقة بدأنا بأول حلقة تكرارية وهي الfor
في هذه المحاضرة سنكمل باقي الحلقات ونتعلم الفرق بينهم.

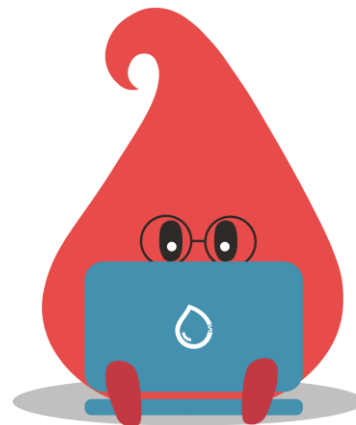
مهمة for while

- for تتميز بمعرفة كم مرة يجب تنفيذ الخطوات.
- جميع الحلقات التكرارية تتميز بإمكانية تنفيذ مجموعة من الخطوات مرتبطة بشرط معين يعبر عن مرات التكرار.

استخدام while:

كتابة برنامج يدخل فيه المستخدم عدداً من الأرقام ويجمعهم ويطبعمهم ويتوقف عند إدخال عدد سالب:

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     int x,sum=0;
6     cin>>x;
7     while(x>=0)
8     {
9         sum=sum+x;
10        cin>>x;
11    }
12    cout<<sum;
13    return 0;
14 }
```





شرح الكود:

في السطر 5 صرحنا عن sum ووضعنا فيها 0 حتى لا يضع البرنامج قيمة عشوائية وبالتالي يتأثر المجموع.

في هذا الكود:

طالما أن $x > 0$ سيضيف البرنامج قيمتها إلى sum ويطلب إدخال x جديدة

عند إدخال x جديدة موجبة ستحذف قيمة x السابقة من الذاكرة وتوضع x الجديدة عوضاً عنها.

في حال إدخال أول x سالبة سيطلع sum=0

سؤال: ماذا لو لم نضع $\text{cin} >> x$ في السطر 10؟

الجواب: ستصبح الحلقة لا نهائية أي ستبقى قيمة x التي أدخلناها أول مرة وسيتم تجاوزها في الشرط عدد لا نهائي من المرات.

استخدام do while:

أعد كتابة البرنامج السابق باستخدام do while:

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     int x=0,sum=0;
6     do{
7         sum=sum+x;
8         cin>>x;
9     }
10    while(x>=0);
11    cout<<sum;
12    return 0;
13 }
```



شرح الكود:

هنا سينفذ الشرط مرة واحدة على الأقل حتى ولو لم يتحقق الشرط.

عرفنا $x=0$ في السطر 4 لنضمن بداية صحيحة للبرنامج.

ملخص الفرق بين
البرامج الثلاثة:

While

طريقة كتابتها:

while(شرط)
{مجموعة أوامر}

وظيفتها: تنفذ مجموعة
أوامر في حال تحقق
الشرط وتتوقف عن
التنفيذ في حال اختل
الشرط.

Do while

طريقة كتابتها:

do
{مجموعة أوامر}
while(شرط);

وظيفتها: تشبه while
ولكن هنا سيتم تنفيذ
الأوامر **مرة على الأقل**
حتى ولو كانت المرة
الأولى معارضة للشرط.

for

طريقة كتابتها:

for(; ;)
{مجموعة أوامر}

وظيفتها:
معرفة عدد مرات تكرار
الأوامر **بدقة**.



تعريف عن العامل:

$$n! = (n)(n-1)(n-2) \dots \times 1$$

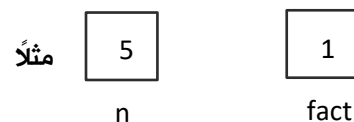
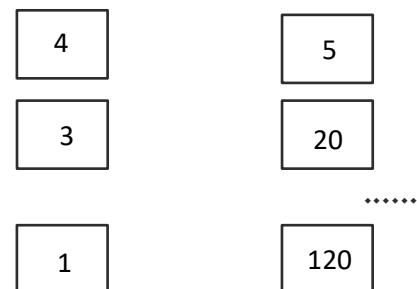
تذكرة:

تمرين: اكتب برنامج يحسب العامل لعدد يتم إدخاله:

```

1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     int n,fact=1;
6     cin>>n;
7     for(int i=n;i>=1;i--)
8         fact=fact*i;
9     cout<<fact;
10    return 0;
11 }
```

شرح الكود:

أولاً يدخل المستخدم قيمة n وتخزن في الذاكرةسيقوم البرنامج بضرب قيمة n ب $fact$ ومن ثم إنقاص n بمقدار 1ثم سيتوقف عندما تصبح قيمة $i=1$ ويطبع الناتج.

تعليمتا continue و break

▪ break :

تستخدم لكسر الحلقة والخروج منها.

مثال توضيحي:

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     for(int i=1;i<=10;i++)
6     {
7         if(i==4)
8             break;
9         cout<<i<<endl;
10    }
11    return 0;
12 }
```



```
OUTPUT
1
2
3
```

التوضيح:

يبدأ البرنامج من $i=1$ وهي $i \neq 4$ فقام بطباعتهاثم $i++=2$ وهي لا تساوي 4 فقام بطباعتها $i++=4$ وهي تحقق الشرط داخل if فتم كسر الحلقة بالكامل وتوقف البرنامج.

Continue

في حال كتابة البرنامج السابق واستبدال break بـ continue في السطر 8، سيكون الخرج:

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     for(int i=1;i<=10;i++)
6     {
7         if(i==4)
8             continue;
9         cout<<i<<endl;
10    }
11    return 0;
12 }
```

OUTPUT

```
1
2
3
5
6
7
8
9
10
```

لم يطبع 4 بعد السطر الثالث.

التوضيح: عند الوصول إلى 4 الذي يحقق شرط if سيتم تجاهله بالانتقال للعدد التالي.



مثال 2: اكتب برنامج يطبع الأعداد الزوجية باستخدام continue:

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     for(int i=1;i<=100;i++)
6     {
7         if(i%2==1)
8             continue;
9         cout<<i<<endl;
10    }
11    return 0;
12 }
```

OUTPUT

```
2
4
6
8
10
12
...
96
98
100
```

البرنامج سيتجاهل الأعداد الفردية ويطبع الزوجية فقط.

مثال 3: اكتب برنامج يطبع التالي:

التفكير البرمجي: سنحتاج for
داخل for لأننا سنطبع سطور
وأعمدة.

```
****
****
****
****
```

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     for(int i=1;i<=3;i++)
6     {
7         for(int j=1;j<=4;j++)
8         {
9             cout<<"*";
10        }
11        cout<<endl;
12    }
13    return 0;
14 }
```

شرح الكود:

i مسؤولة عن الأسطر

z مسؤولة عن الأعمدة

سنبدأ البرنامج بالسطر الأول i=1 ثم العمود الأول z=1 فيطبع:

*

ثم z=2 فيطبع:

**

عندما z=5 يصبح:

تكسر حلقة z ثم تصبح i=2 وتكرر العملية نفسها حتى تصبح i=4 فينتهي البرنامج ويطبع الشكل المطلوب.
في حال لم تستعمل cout<<endl; في السطر 11 سيظهر 12 نجمة في سطر واحد.

المصفوفات

وهي Data type يستعمل عند الحاجة إليه بحيث يحجز في الذاكرة عدد معين من العناصر ومن النوع نفسه حصراً.

مثال:

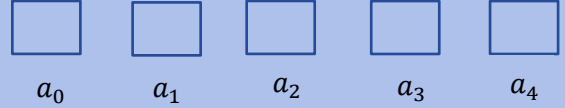
اكتب برنامج لطباعة عناصر تخزين ضمن مصفوفة:

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     int n;
6     cin>>n;
7     int a[n];
8     for(int i=0;i<n;i++)
9         cout<<a[i];
10    return 0;
11 }
```



شرح الكود:

البرنامج خزن n عنصراً ضمن مصفوفة تسمى a ولنفرض $n=5$



يكون آخر رقم يعبر عن المصفوفة الأخيرة $n-1$

أول عنصر a_0

آخر عنصر a_{n-1}

يدخل المستخدم أولاً قيمة a_0 فيتم تخزينها ضمن a_0

مثلاً أدخل 10

10

 a_0

سؤال: ماذا لو كتبنا البرنامج التالي، ما قيمة الخرج؟

```

1 #include <iostream>
2 using namespace std;
3 int main()
4 { int b[5]={3,8,9,1,6};
5   cout<<b[5];
6   return 0;
7 }
```

بما أن b تحوي 5 عناصر فقط وآخر عنصر اسمه $b[4]$ فإن $b[5]$ غير موجودة في المصفوفة.

سيقوم البرنامج بعمل غير متوقع:

إما error

أو سيعطي قيمة عشوائية

وذلك حسب مكان المصفوفة في الذاكرة.



أما إذا كان `cout<<b[3]`

سيكون الخرج 1

لأن العنصر الثالث من المصفوفة مخزن فيه 1

انتهت المحاضرة الرابعة 😊

